

Лабораторная работа

Тема: Аналитическая колоночная СУБД ClickHouse: колоночное хранение, разреженный первичный индекс, асинхронные мутации

1. Цель работы

Изучить ключевые особенности ClickHouse, которые влияют на производительность аналитических запросов:

- как колоночное хранение ускоряет чтение только нужных столбцов;
- почему разреженный первичный индекс позволяет быстро фильтровать данные без точной привязки к каждой строке;
- зачем нужны асинхронные мутации вместо обычных **UPDATE/DELETE** и как они работают.

Научиться создавать таблицы семейства MergeTree, загружать большие объёмы данных, анализировать планы запросов и применять мутации.

2. Ход работы и результаты

Задача 1. Подготовка окружения и создание таблицы

1. Запущен ClickHouse в Docker-контейнере.

```
C:\Users\Святослав>docker run -d --name clickhouse-server --ulimit nofile=262144:262144 yandex/clickhouse-server
8630f58efd490a3215ac3684fb647bf30c6cf4a77fc2e39e59cf0a7845570fda

C:\Users\Святослав>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
8630f58efd49   yandex/clickhouse-server           "/entrypoint.sh"        12 seconds ago Up 11 seconds  8123/tcp, 9000/tcp, 9009/t
cp   clickhouse-server

C:\Users\Святослав>docker exec -it clickhouse-server clickhouse-client
ClickHouse client version 22.1.3.7 (official build).
Connecting to localhost:9000 as user default.
Connected to ClickHouse server version 22.1.3 revision 54455.
```

2. Выполнен DDL для создания таблицы **test_events** типа **MergeTree**:

```
CREATE TABLE test_events
(
    event_date Date,
    user_id UInt32,
    event_type LowCardinality(String),
    value Float32
)
ENGINE = MergeTree
ORDER BY (event_date, user_id);
```

3. Проверка создания таблицы:

```
SHOW CREATE TABLE test_events;
```

```
8630f58efd49 :) CREATE TABLE test_events (  
    event_date Date,  
    user_id UInt32,  
    event_type LowCardinality(String),  
    value Float32  
    ) ENGINE = MergeTree()  
    ORDER BY (event_date, user_id);
```

```
CREATE TABLE test_events  
(  
    `event_date` Date,  
    `user_id` UInt32,  
    `event_type` LowCardinality(String),  
    `value` Float32  
)  
ENGINE = MergeTree  
ORDER BY (event_date, user_id)
```

Query id: 4a8d99c5-181f-4522-952d-6e1f2662a483

Ok.

0 rows in set. Elapsed: 0.011 sec.

```
8630f58efd49 :) SHOW CREATE TABLE test_events
```

```
SHOW CREATE TABLE test_events
```

Query id: 59bb6786-71cb-46f5-8dcd-66ce7f787fe8

```
┌statement┐  
└────────┘  
| CREATE TABLE default.test_events  
| (  
|     `event_date` Date,  
|     `user_id` UInt32,  
|     `event_type` LowCardinality(String),  
|     `value` Float32  
| )  
| ENGINE = MergeTree  
| ORDER BY (event_date, user_id)  
| SETTINGS index_granularity = 8192 |  
└────────┘
```

Пояснение:

- `event_date` – дата события (используется в первичном ключе);
- `user_id` – идентификатор пользователя;
- `event_type` – тип события, с модификатором `LowCardinality` (эффективное хранение повторяющихся строк);
- `value` – числовое значение (например, сумма покупки);
- `ORDER BY (event_date, user_id)` – задаёт ключ сортировки и неявно – разреженный первичный индекс.

Задача 2. Загрузка данных

Вставлено 300 000 строк с помощью генерации случайных значений:

```
INSERT INTO test_events
SELECT
    today() - rand() % 365 AS event_date,
    rand() % 100000 AS user_id,
    arrayElement(['view', 'click', 'purchase', 'add_to_cart'], rand() % 4) AS
event_type,
    rand() % 10000 / 10 AS value
FROM numbers(300000);
```

Проверка количества строк:

```
SELECT count() FROM test_events;
```

```
8630f58efd49 :) INSERT INTO test_events
SELECT
    today() - rand() % 365 AS event_date,
    rand() % 100000 AS user_id,
    arrayElement(['view', 'click', 'purchase', 'add_to_cart'], rand() % 4) AS event_type,
    rand() % 10000 / 10 AS value
FROM numbers(300000);

INSERT INTO test_events SELECT
    today() - (rand() % 365) AS event_date,
    rand() % 100000 AS user_id,
    ['view', 'click', 'purchase', 'add_to_cart'][rand() % 4] AS event_type,
    (rand() % 10000) / 10 AS value
FROM numbers(300000)

Query id: 12c8c560-f316-45c6-839b-88a02e09109c
Ok.
0 rows in set. Elapsed: 0.114 sec. Processed 300.00 thousand rows, 2.40 MB (2.63 million rows/s., 21.03 MB/s.)

8630f58efd49 :) SELECT count() FROM test_events;

SELECT count()
FROM test_events

Query id: f99b971d-3fd8-49de-a6b6-ae054b21363d

count()
300000

1 rows in set. Elapsed: 0.002 sec.
```

Результат: 300 000 строк загружены.

Задача 3. Колоночное хранение

Цель: сравнить скорость чтения всех колонок (`SELECT *`) и только двух колонок (`SELECT event_date, sum(value)`).

Запрос 1 (читает все колонки):

```
SELECT * FROM test_events WHERE event_date = '2025-06-15';
```

Время выполнения:

2025-06-15	99204		920.4
2025-06-15	99284		928.4
2025-06-15	99344		934.4
2025-06-15	99409	view	940.9
2025-06-15	99654	click	965.4
2025-06-15	99674	click	967.4

830 rows in set. Elapsed: 0.008 sec. Processed 8.19 thousand rows, 90.17 KB (1.06 million rows/s., 11.65 MB/s.)

(по данным из вывода клиента)

Запрос 2 (читает только `event_date` и `value`):

```
SELECT event_date, sum(value) FROM test_events WHERE event_date = '2025-06-15'
GROUP BY event_date;
```

Время выполнения:

8630f58efd49 :) SELECT event_date, sum(value) FROM test_events WHERE event_date = '2025-06-15' GROUP BY event_date;

```
SELECT
  event_date,
  sum(value)
FROM test_events
WHERE event_date = '2025-06-15'
GROUP BY event_date
```

Query id: f9090f0d-3d98-48a0-8169-df78a13f11ad

event_date	sum(value)
2025-06-15	436809.12929844856

1 rows in set. Elapsed: 0.004 sec. Processed 8.19 thousand rows, 49.15 KB (2.07 million rows/s., 12.42 MB/s.)

Вывод:

Второй запрос выполняется быстрее, потому что ClickHouse читает с диска только файлы, соответствующие колонкам `event_date` и `value`, и не тратит время на чтение колонок `user_id` и `event_type`. Это прямое следствие колоночного хранения данных.

Задача 4. Разреженный первичный индекс**Запрос с фильтром по ключу (быстрый):**

```
SELECT count() FROM test_events WHERE event_date = '2025-06-15';
```

Время:

8630f58efd49 :) SELECT count() FROM test_events WHERE event_date = '2025-06-15';

```
SELECT count()
FROM test_events
WHERE event_date = '2025-06-15'
```

Query id: 8d41c2ee-3623-4300-a64f-f2e725fdd539

count()
830

1 rows in set. Elapsed: 0.003 sec. Processed 8.19 thousand rows, 16.38 KB (2.83 million rows/s., 5.67 MB/s.)

Запрос с фильтром по неключевой колонке (медленный):

```
SELECT count() FROM test_events WHERE value > 50000;
```

Время:

```
8630f58efd49 :) SELECT count() FROM test_events WHERE value > 50000;

SELECT count()
FROM test_events
WHERE value > 50000

Query id: 0488cd60-31d3-4ef5-a895-000a81ee26d1

count()
0

1 rows in set. Elapsed: 0.047 sec. Processed 300.00 thousand rows, 1.20 MB (6.37 million rows/s., 25.46 MB/s.)
```

Задача 5. Асинхронные мутации

1. Проверка наличия строк с `event_type = 'purchase'`:

```
8630f58efd49 :) SELECT event_type, count() FROM test_events GROUP BY event_type;

SELECT
  event_type,
  count()
FROM test_events
GROUP BY event_type

Query id: 40b07e66-614c-4a90-8107-cb8491dea600

event_type  count()
click       74949
view        74977
purchase    75136
purchase    74938

4 rows in set. Elapsed: 0.047 sec. Processed 300.00 thousand rows, 300.26 KB (6.33 million rows/s., 6.34 MB/s.)
```

2. Запуск мутации (увеличиваем `value` на 10% для всех покупок):

```
ALTER TABLE test_events UPDATE value = value * 1.1 WHERE event_type = 'purchase';
```

3. Сразу после отправки команды проверяем состояние мутации:

```
SELECT mutation_id, is_done, latest_failed_part, latest_fail_time
FROM system.mutations
WHERE table = 'test_events'
FORMAT Vertical;
```

```

8630f58efd49 :) ALTER TABLE test_events UPDATE value = value * 1.1 WHERE event_type = 'purchase';
ALTER TABLE test_events
  UPDATE value = value * 1.1 WHERE event_type = 'purchase'
Query id: d52be7f4-53cb-4748-bdd2-3db7e906749c
Ok.
0 rows in set. Elapsed: 0.027 sec.

8630f58efd49 :) SELECT mutation_id, is_done, latest_failed_part, latest_fail_time
                  FROM system.mutations
                  WHERE table = 'test_events'
                  FORMAT Vertical;

SELECT
  mutation_id,
  is_done,
  latest_failed_part,
  latest_fail_time
FROM system.mutations
WHERE table = 'test_events'
FORMAT Vertical
Query id: d4e66ccf-6274-4d09-a397-dee9916ffd76
Row 1:
mutation_id:      mutation_2.txt
is_done:          1
latest_failed_part:
latest_fail_time:  1970-01-01 00:00:00
1 rows in set. Elapsed: 0.003 sec.

```

ClickHouse не перезаписывает строки на лету. В момент выполнения **ALTER TABLE ... UPDATE** создаётся запись в **system.mutations**, а фоновый процесс переписывает затронутые части данных (**parts**) в новые файлы. Во время этого процесса старые данные остаются доступными для чтения. Поэтому мутации асинхронны и не блокируют запросы **SELECT**.

Задача 6. Сводка особенностей и аналогия

Три особенности и их влияние:

1. **Колоночное хранение** – при аналитическом запросе ClickHouse читает только нужные столбцы с диска, что резко сокращает объём ввода-вывода и ускоряет агрегации.
2. **Разреженный первичный индекс** – позволяет отсекают целые группы строк (гранулы) без просмотра каждой строки, давая очень быстрый поиск по ключам при минимальном размере индекса.
3. **Асинхронные мутации** – делают команды **UPDATE/DELETE** неблокирующими для чтения; изменения применяются в фоне, что поддерживает высокую пропускную способность системы.

Педагогическая аналогия (для студентов):

Разреженный первичный индекс в ClickHouse похож на **оглавление в книге, где указаны только главы, а не номера страниц**.

Если вы ищете тему, которая точно находится в главе 3, вы открываете её целиком, но не листаете каждую страницу от начала до конца.

А плотный индекс (как в обычных БД) — это список каждой страницы, который занимает слишком много места.

Благодаря «оглавлению по главам» ClickHouse быстро находит нужные группы строк (гранулы) и тратит гораздо меньше памяти на индекс.

3. Общие выводы

ClickHouse принципиально отличается от традиционных реляционных СУБД (например, PostgreSQL или MySQL) следующими чертами:

- Ориентация на **аналитические запросы** (OLAP) с большими объёмами данных, а не на транзакционную обработку (OLTP).
- **Колоночное хранение** данных, которое позволяет эффективно сжимать однотипные значения и читать только необходимые столбцы.
- **Отказ от немедленных UPDATE/DELETE** в пользу асинхронных мутаций – это плата за скорость агрегаций на миллиардах строк.
- **Разреженные индексы** вместо плотных – из-за гранулярной организации данных (гранулы) индекс остаётся маленьким, но эффективным для фильтрации по первичному ключу.

Лабораторная работа показала, что даже на небольшом объёме данных (300 тыс. строк) уже заметен выигрыш от колоночного чтения и разреженного индекса. Асинхронный характер мутации был наглядно подтверждён через системную таблицу `system.mutations`.